BOXSPLOIT LABS / PRODUCT DOCUMENTATION

PANBOX

Product & operator guide

Self-hosted AppSec control plane. Setup, CI connectivity, access control, operations, and honest pilot limits.

VERSION 0.1 · 17 OCTOBER 2026 · PILOT

© 2026 BOXSPLOIT LABS

Contents

1. What runs where

2. Choose your setup path

3. Start the dashboard on Debian/Linux/macOS

4. Start the dashboard on Windows

5. Add only the agent to CI

6. Reach private PANBOX from public SCM

7. Give people only needed access

8. Manage API and access keys

9. Confirm and control scans in the dashboard

10. Operate within the pilot boundary

11. Troubleshoot by failed step

12. Know the alpha limits

This guide describes implemented pilot behavior. Roadmap items are labelled separately.

© 2026 BOXSPLOIT LABS

Start with the path that matches your job: server operator installs the private dashboard once; DevOps adds an agent-only step to each CI job. The agent is one Go binary. The runner does not need Node, Python, Semgrep, Grype, or a PANBOX server installation.

1. What runs where

```
Git host/CI -> protected runner -> PANBOX agent -> private PANBOX API -> PostgreSQL
                                  | VACCIN: source rules            |
                                  | ENETRON: packages -> OSV        +-> React UI
```

Result: CI receives pass, gate failure, incomplete, or authorization failure; operators see repository and branch snapshots. Source files stay on the runner. ENETRON sends package names and versions to an OSV-compatible advisory API. The agent uploads normalized findings with a PANBOX API key, without raw source snippets. See the architecture and network diagram.

2. Choose your setup path

- Server operator, once: Docker Engine/Desktop with Linux containers and Compose v2. Start with section 3 on Debian/Linux/macOS or section 4 on Windows. Debian is the initial production target.
- DevOps, per repository: enroll a repository, create one API key with scans:write, set CI secrets, and run section 5. The runner needs outbound HTTPS to GitHub Releases and the advisory API, plus a route to the private PANBOX API.
- No public IP/domain: choose a route in section 6 before writing the CI job.

PANBOX binds HTTP 8080 and PostgreSQL 5434 to host loopback by default. Change PANBOX_HOST_PORT or PANBOX_DB_HOST_PORT in .env if needed. Keep PostgreSQL private.

3. Start the dashboard on Debian/Linux/macOS

Goal: bring up the private API and UI once. Run on the server host:

```
git clone https://github.com/alterwriter/PANBOX.git
cd PANBOX
sh scripts/init-local.sh
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
curl -fsS http://127.0.0.1:8080/readyz
```

Expected: /readyz succeeds. Open http://127.0.0.1:8080 on the server and sign in with PANBOX_BOOTSTRAP_EMAIL and PANBOX_BOOTSTRAP_PASSWORD from the ignored .env. The initializer creates secrets with restricted permissions and refuses to overwrite an existing .env. Bootstrap credentials apply only when the database is first initialized; retain its volume on routine restarts.

Optional demo: PANBOX_SERVER=http://127.0.0.1:8080 bash scripts/seed-demo.sh needs curl and jq, then prompts for the admin password. Demo findings are synthetic.

4. Start the dashboard on Windows

Goal: run the same Linux containers through Docker Desktop. In PowerShell:

```
git clone https://github.com/alterwriter/PANBOX.git
cd PANBOX
./scripts/init-local.ps1
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
Invoke-RestMethod http://127.0.0.1:8080/readyz
```

Expected: the same dashboard and API as section 3. Use ./scripts/seed-demo.ps1 only for optional demo data. Run one initializer for a new .env, not both.

© 2026 BOXSPLOIT LABS

5. Add only the agent to CI

Onboarding model: Enroll a PANBOX project before its first scan. Its project ID routes the upload but is not a secret. One API key with scans:write authenticates policy fetch and upload. Private SCM repositories do not need to be made public: CI's checkout credentials fetch source, while PANBOX receives only normalized results.

Once per repository, before changing the pipeline:

1. Sign in as an admin or a user with repos:write, then open Repositories -> Enroll repository. Select GitHub, GitLab, Bitbucket, Azure Repos, Gitea, or Other Git.

2. Enter name (display label, for example PANBOX) and SCM repository ID or path (for example alterwriter/PANBOX; GitLab may use group/subgroup/project). Select Test SCM connectivity and detect branch. Public metadata from GitHub, GitLab, Bitbucket, Azure Repos, or an operator-allowed Gitea host fills default branch automatically. Owner may be blank; it is display metadata, not RBAC or scan authorization. Display name, SCM path, and internal PANBOX ID are three different values.

3. Read the result before selecting Enroll repository. Verified metadata means the PANBOX host read the default branch. Waiting for CI means private/unreachable metadata; keep Auto-detect selected. The first authenticated CI upload can supply the true default branch, even when it scans a feature branch. PANBOX never guesses the first scanned branch is default. For self-hosted Git, add the exact HTTPS host to PANBOX_SCM_ALLOWED_HOSTS. Copy PANBOX ID for CI to PANBOX_REPOSITORY_ID; this internal ID differs from the SCM path or GitLab project ID.

4. Select Create CI API key. The admin/key manager needs tokens:manage, scans:write, and project access. Keep scans:write, optionally scope the key to this PANBOX project, and save the one-time secret as protected CI secret PANBOX_API_KEY. No separate repository token is needed. The key owner still needs project membership unless admin.

5. Open the row's Scan policy: enable scans, select VACCIN/ENETRON, and set the gate threshold. Set PANBOX_SERVER in CI to the API URL reachable from the runner, such as PANBOX's LAN address. 127.0.0.1 works only when runner and server share a host. Never put an API key in repository files or untrusted pull request jobs.

6. From the runner using its CI checkout credentials, verify checkout succeeded (git rev-parse --is-inside-work-tree), then check dashboard reachability, key, and ID with the commands below. The dashboard metadata check in step 2 does not prove either runner path. Once both succeed, run the agent job below. The first complete scan appears in Scans, with its findings in Findings.

```
curl -fsS "$PANBOX_SERVER/readyz"
curl -fsS -H "Authorization: Bearer $PANBOX_API_KEY" \
  "$PANBOX_SERVER/v1/repos/$PANBOX_REPOSITORY_ID/scan-config"
```

Branches after enrollment: every job reports its checkout branch, so a feature branch appears separately once CI runs PANBOX there. GitLab CI_DEFAULT_BRANCH and the GitHub template's PANBOX_DEFAULT_BRANCH report the actual default even from a feature branch. Other CI systems can set PANBOX_DEFAULT_BRANCH or use --default-branch. Without this metadata, default stays pending. The supplied GitLab job runs on protected branches; extend CI rules for others. Overview uses the default branch baseline. Recheck SCM refreshes auto-detected metadata.

Fix a wrong enrollment: click a repository row to open details. Edit changes display name, owner, and default branch; select Manual override when needed. Choose Self-hosted / enterprise and enter an HTTPS origin for GitHub Enterprise or GitLab self-managed; allowlist its exact host in PANBOX_SCM_ALLOWED_HOSTS. SCM identity can be edited until the first scan, then it is locked to preserve finding history. Admin Remove deletes an unscanned repository or archives a scanned one and revokes project-bound keys. Admin can Restore and use an API key with project access. A missing owner is safe: it never grants permission.

Custom native VACCIN rule: commit .panbox/rules.json to the scanned repository. PANBOX reads it on the runner before scanning, alongside built-in rules; an invalid file fails the VACCIN module rather than silently passing. Example:

```
{"rules":[{"id":"custom.team.unsafe-call","severity":"medium","message":"Unsafe call needs
review","pattern":"unsafeCall\\(","extensions":[".js"]}]}
```

The JSON pattern above represents regex unsafeCall\(. IDs start with custom.; supported extensions are .py, .js, .jsx, .ts, .tsx, and .go. Limit: 64 rules, 256 characters per pattern, 64 KiB file. Rules operate on individual lines after basic comment/string stripping; they are not taint analysis. Keep the rule file under normal code review and test changes on representative repos.

Every Linux/macOS job: download the pinned installer from PANBOX release v0.1.0-alpha.6. It selects OS/CPU, downloads just the binary, checks SHA256SUMS, and installs in the job workspace without sudo:

```
PANBOX_VERSION=v0.1.0-alpha.6
curl -fsSL
"https://github.com/alterwriter/PANBOX/releases/download/${PANBOX_VERSION}/install-agent.sh" -o
install-panbox-agent.sh
sh install-panbox-agent.sh "$PANBOX_VERSION" .panbox/bin
.panbox/bin/panbox go --server "$PANBOX_SERVER" --repo "$PANBOX_REPOSITORY_ID" --path .
```

Expected: exit 0 complete/pass; 1 gate failure; 2 incomplete/transport/config error; 3 rejected auth/permission. Copy the GitHub Actions job or GitLab CI job. On Windows, download install-agent.ps1 and run ./install-agent.ps1 -Version v0.1.0-alpha.6; it installs ./.panbox/bin/panbox.exe after checksum verification. Direct agent-only binaries are also available.

VACCIN checks selected Python, JavaScript/TypeScript, and Go source patterns. ENETRON matches pinned npm lockfiles v2/v3, requirements.txt, and go.mod dependencies against OSV advisory data. OSV supplies data, not scanner execution. PANBOX_ADVISORY_URL or --advisory-url selects an approved OSV-compatible mirror. --offline suppresses dashboard upload, but ENETRON still contacts the advisory API unless only --modules vaccin is selected.

CI-to-dashboard sequence: (1) the runner downloads the pinned agent; (2) panbox go authenticates GET /v1/repos/{id}/scan-config with the CI API key; (3) the agent scans enabled modules in the checkout; (4) it posts one normalized JSON bundle to POST /v1/scans with Authorization: Bearer $PANBOX_API_KEY; (5) the API stores the snapshot and returns the gate verdict; (6) the agent exits 0 (pass), 1 (gate fail), 2 (incomplete/transport), or 3 (auth). There is no separate upload setup. Verify the policy from the runner with curl -fsS -H "Authorization: Bearer $PANBOX_API_KEY" "$PANBOX_SERVER/v1/repos/$PANBOX_REPOSITORY_ID/scan-config".

6. Reach private PANBOX from public SCM

- Local protected runner (recommended): polls any SCM over outbound HTTPS and posts to PANBOX on its LAN. No public PANBOX port; synchronous gate. Use the GitHub or GitLab example.
- SSH relay: PANBOX opens an outbound reverse tunnel to a reachable relay; a shared runner opens a forward tunnel. No PANBOX inbound internet port; synchronous gate. Pin host keys and restrict identities. See the GitLab relay example.
- Artifact import: CI writes panbox-results.json; a private host fetches and imports it later. No inbound route or relay; the later gate cannot block that CI job. GitLab artifact example and panbox relay-pull automate GitLab; other SCMs need their artifact download step before generic panbox go --bundle import.

Check: from a synchronous runner, curl -fsS "$PANBOX_SERVER/readyz" must work. A public repository does not make its CI artifacts public.

7. Give people only needed access

Admin setup: Settings & Audit -> RBAC -> create a role with explicit permissions -> assign users -> grant repository membership. admin is the highest immutable role. Built-in roles include security_reviewer, developer, executive_viewer, and scanner_ci; custom roles cannot receive rbac:manage.

Check: data access needs both permission and repository membership. Pilot permissions are repos:read, repos:write, scans:read, scans:write, findings:read, findings:recast, reports:export, rbac:manage, tokens:manage,

and audit:read.

8. Manage API and access keys

Where: Settings & Audit -> API & access keys. Choose label, type, expiry, scopes, and repository when applicable. API keys carry selected integration scopes; access keys are read-only; CI API keys use scans:write and optional project scope. Copy the one-time secret into a secret manager. Keys are hashed at rest, expiring, and revocable.

Read-only smoke test:

```
curl -H "Authorization: Bearer $PANBOX_ACCESS_KEY" "$PANBOX_SERVER/v1/repos"
```

Browser login uses password, HttpOnly session cookie, and CSRF token. The OpenAPI contract lists pilot endpoints. Use Rotate to issue a replacement and revoke the old key atomically; update the CI secret before its next job.

9. Confirm and control scans in the dashboard

Investigate and triage: click a repository row for its details and direct View findings / View scans actions. Click a scan to see its findings, then click a finding for its path, line, source URL when a safe commit link can be formed, advisory, remediation guidance, and the last 100 eligible complete scans on that branch. An absent finding on a later complete scan of its module appears as resolved; earlier history remains visible. Recast severity changes effective severity with an audited reason. Triage can mark false positive, accepted risk, closed, or reopen, also with a reason; these statuses revise affected gates and remove the finding from active posture counts while retaining it in history and reports. Closure is manual and persists until reopened. The remediation prompt button copies a local template; it does not call AI or modify source. The read-only MCP bridge can supply findings to a client, but the client controls any AI use. No false-positive percentage has been measured for this pilot.

Recast: open Findings, clear filters if no rows appear, then select Recast severity beside a finding rule (the first column). Choose an effective severity or restore the scanner value -> enter a reason (10-500 characters) -> Save. The button appears only with findings:recast and repository access; an admin or security_reviewer with repository membership can use it. If no rows appear, complete the first scan or check filters. If rows appear without the button, ask an admin to check role and membership under Settings & Audit -> RBAC. Raw severity never changes. The API exposes history at GET /v1/findings/{id}/recasts; affected historic gate revisions are recalculated against each scan's original threshold. Recast is not an approval or dismissal.

Export: Reports -> choose repository, exact branch, scan module (VACCIN/ENETRON), and optional severity -> Export PDF, CSV, or JSON. Data comes from the latest complete scan per branch. PDF is a readable technical summary; CSV/JSON retain full rows. Requires reports:export.

MCP: Settings & Audit -> API & access keys -> create a read-only Access key with repos:read, scans:read, and findings:read. Open Settings & Audit -> MCP setup and use its generic stdio host configuration. The MCP host launches panbox mcp with PANBOX_SERVER and PANBOX_TOKEN in its secret environment. Four read-only tools list repositories, findings, scans, and scan detail. There is no public MCP endpoint.

After CI: Overview shows coverage and severity; Repositories shows enrolled projects; Scans shows run/gate status; Findings filters by repository/branch/module/severity; Reports exports filtered technical PDF/CSV/JSON; Settings & Audit shows RBAC, keys, and audit events. Light and dark themes are available.

Interpretation: a partial scan is unverified and preserves the last successful posture. ARK labels Critical/High and BUG-STER labels Medium/Low while standard severities remain stored and exported. Recast actions and changes to scan policy are audited.

10. Operate within the pilot boundary

Keep PostgreSQL on loopback. For remote browser access, use an HTTPS reverse proxy with access control or a private route; set PANBOX_SECURE_COOKIE=true for HTTPS. Back up PostgreSQL and test restoration.

© 2026 BOXSPLOIT LABS

The 2 GB Debian host is a target to validate, not a guarantee. A small Windows Docker Desktop fixture measured about 5.1 MiB server RAM and 37.3 MiB PostgreSQL RAM; 100 sequential API calls measured 7.1 ms median and 9.3 ms p95. The agent runs on the runner and needs separate sizing. Measure docker stats --no-stream with representative repositories and scan bundles.

11. Troubleshoot by failed step

- Dashboard not ready: run docker compose --env-file .env -f deploy/docker-compose.yml ps and logs db server; confirm PostgreSQL health.
- Agent download fails: check release tag, OS/CPU asset name, outbound GitHub access, curl, and SHA-256 tooling. Do not skip the checksum.
- Runner cannot reach PANBOX: 127.0.0.1 inside a container is that container; use its reachable private host/network address or the SSH relay.
- Exit 2: check manifest, advisory endpoint, work path, PANBOX endpoint, and logs; incomplete is not pass.
- Exit 3: check API key expiry/revocation, scans:write, internal repository ID, role, and membership.
- Artifact import is late: check timer, GITLAB_READ_TOKEN, allowed refs, Jobs API/artifact access, and relay state file.

12. Know the alpha limits

Available now: server/UI, native VACCIN source checks, native ENETRON pinned dependency matching with OSV data, repository/branch snapshots, gate, RBAC, scoped keys, audit, filtered PDF/CSV/JSON exports, and the transport examples above. Validate real runners, OSV access or approved mirror, Debian load, backup/restore, TLS, and key rotation in your environment.

Coverage is intentionally narrow. RIDER secrets, TESLA IaC, approval workflows, executive PDF templates, AI Security, OIDC, and API rate limiting are roadmap items. This PDF is a product guide, not a PANBOX vulnerability report.

© 2026 BOXSPLOIT LABS